

MODELO PADRÃO DE ENTREGA DE ATIVIDADES

Etapas	Entrega final	Atividade	9
Encontro	10	Curso	ADS / ULBRA
Aluno	Vinicius Casagrande	Professor(a)	Caroline Pacheco da Rosa
Projeto	Recomendação de treinos personalizados em academia		

Artefato verificável

Repositório (GitHub): https://github.com/Vinicasagrande87/fundamentos_ia

Notebook desta etapa: **atividade9_academia.ipynb** (14 seções, executado do início ao fim no Google Colab antes do envio)

README.md atualizado: seção "9. Atividade 9 — Primeiro modelo treinado (entrega final)" e backlog do projeto, com link do notebook e do resultado.

1. Descrição do problema

O projeto busca recomendar a categoria de treino mais adequada para um aluno de academia a partir do seu perfil. A saída desejada é uma categoria de treino, e a entrada contém informações como idade, objetivo, nível de experiência, disponibilidade de dias por semana, restrição física e equipamento disponível.

O problema foi tratado como classificação supervisionada multiclasse, porque o objetivo é prever uma categoria dentre várias possibilidades: Treino A, Treino B, Treino C ou Treino D.

2. Base tratada

A base tratada da Atividade 8 foi carregada e utilizada neste notebook. Antes do treino, a coluna identificadora `aluno_id` foi removida das features, pois ela não contribui para a aprendizagem do padrão do problema.

Variável alvo: `treino_recomendado`

Features: `idade`, `objetivo`, `nivel_experiencia`, `dias_disponiveis_semana`, `restricao_fisica`, `equipamento_disponivel`

A base foi separada em X (features) e y (alvo).

3. Expectativa inicial registrada antes do treinamento

Antes de treinar o modelo, a expectativa era um desempenho acima do baseline, mas sem expectativa exagerada: a base tem padrões relativamente claros entre o perfil do aluno e a recomendação do treino, mas também há ruído e exceções. Por isso, foi considerada plausível uma acurácia entre 60% e 80%.

4. Divisão treino e teste

A divisão foi feita com 80% para treino e 20% para teste, usando `random_state=42` e `stratify=y`, para manter a proporção das classes em treino e teste.

- `X_train`: 240 linhas · `X_test`: 60 linhas
- `y_train`: 240 valores · `y_test`: 60 valores

5. Modelo baseline treinado

Baseline: `DummyClassifier(strategy="most_frequent")`, que sempre responde a classe mais frequente.

Acurácia do baseline: 0,30

6. Escolha do modelo e justificativa (consolidada)

O modelo escolhido — Árvore de Decisão (DecisionTreeClassifier, max_depth=5, random_state=42) — foi mantido em relação ao esboço; não houve troca de modelo.

- o problema é de classificação multiclasse;
- a base possui poucas variáveis e a maioria delas é categórica;
- a árvore é interpretável, o que permitiu, nesta entrega final, extrair as regras aprendidas (export_text) e compará-las diretamente com a regra de geração da base (seção 9), reforçando a justificativa original;
- é um modelo clássico, simples e adequado para esta etapa inicial do projeto.

Acurácia do modelo: 0,90

7. Previsões geradas

Amostra das previsões do modelo comparadas com os valores reais do conjunto de teste (10 primeiras linhas do notebook):

- real: Treino B | prevista: Treino B
- real: Treino A | prevista: Treino A (6 ocorrências na amostra)
- real: Treino C | prevista: Treino C
- real: Treino D | prevista: Treino D

Lista completa das 60 previsões do conjunto de teste disponível na seção 7 do notebook.

8. Matriz de confusão e F1 por classe

Adicionada nesta entrega final porque a acurácia agregada pode esconder o desempenho de uma classe específica — as classes não são do mesmo tamanho (Treino D: 92, Treino C: 80, Treino A: 74, Treino B: 54 no total da base).

Resultado real (executado no Google Colab):

Matriz de confusao (linhas = real, colunas = previsto):

	prev_Treino A	prev_Treino B	prev_Treino C	prev_Treino D
real_Treino A	15	0	0	0
real_Treino B	1	10	0	0
real_Treino C	1	0	15	0
real_Treino D	2	1	1	14

	precision	recall	f1-score	support
Treino A	0.79	1.00	0.88	15
Treino B	0.91	0.91	0.91	11
Treino C	0.94	0.94	0.94	16
Treino D	1.00	0.78	0.88	18
accuracy			0.90	60
macro avg	0.91	0.91	0.90	60
weighted avg	0.91	0.90	0.90	60

Leitura: o Treino D e a classe mais difícil (recall 0,78, a menor das quatro) e é a que mais se confunde com as demais; as outras três classes têm recall e F1 acima de 0,90.

9. Regras aprendidas pela árvore (interpretabilidade)

A árvore é interpretável, o que permite extrair as regras aprendidas (export_text) e compará-las com a lógica usada para montar o problema (função recomenda_treino em gerar_dataset.py), em vez de tratar o modelo como caixa-preta.

```
|--- objetivo_Emagrecimento <= 0.50
|   |--- objetivo_Hipertrofia <= 0.50
|   |   |--- restricao_fisica_Nenhuma <= 0.50
|   |   |   |--- nivel_experiencia_Iniciante <= 0.50
|   |   |   |   |--- restricao_fisica_Lombar <= 0.50 --> Treino C
|   |   |   |   |--- restricao_fisica_Lombar > 0.50 --> Treino D
|   |   |   |--- nivel_experiencia_Iniciante > 0.50
|   |   |   |   |--- dias_disponiveis_semana <= 2.50 --> Treino A
|   |   |   |   |--- dias_disponiveis_semana > 2.50 --> Treino D
|   |   |--- restricao_fisica_Nenhuma > 0.50
```

```

| | | |--- equipamento_disponivel_Nenhum <= 0.50 --> Treino C
| | | |--- equipamento_disponivel_Nenhum > 0.50 --> Treino D
| | | |--- objetivo_Hipertrofia > 0.50
| | | |--- restricao_fisica_Nenhuma <= 0.50 --> Treino D ou C (por idade)
| | | |--- restricao_fisica_Nenhuma > 0.50
| | | |--- equipamento_disponivel_Nenhum <= 0.50 --> Treino A
| | | |--- equipamento_disponivel_Nenhum > 0.50
| | | |--- dias_disponiveis_semana <= 5.50 --> Treino D
| | | |--- dias_disponiveis_semana > 5.50 --> Treino C
|--- objetivo_Emagrecimento > 0.50
| |--- dias_disponiveis_semana <= 2.50 --> Treino D ou B (por idade/equipamento)
| |--- dias_disponiveis_semana > 2.50
| | |--- restricao_fisica_Nenhuma <= 0.50 --> Treino B ou D
| | |--- restricao_fisica_Nenhuma > 0.50
| | |--- equipamento_disponivel_Nenhum <= 0.50 --> Treino B
| | |--- equipamento_disponivel_Nenhum > 0.50 --> Treino D

```

(arvore completa, com export_text, disponivel na secao 9 do notebook)

Comparacao com a regra de geracao (gerar_dataset.py): as regras aprendidas reproduzem o nucleo da logica real — o primeiro corte por objetivo e o papel de equipamento_disponivel_Nenhum como forte preditor de Treino D aparecem na maioria dos ramos, assim como restricao_fisica_Nenhuma controlando o restante da arvore, no mesmo espirito da funcao recomenda_treino. A correspondencia, porem, nao e perfeita: no ramo objetivo=Hipertrofia, sem restricao, sem equipamento e dias>5,5, a arvore preve Treino C, enquanto a regra real sempre impoe Treino D quando o equipamento e Nenhum — sinal de que, nesse ramo com poucos exemplos de treino, o modelo aprendeu um padrao de ruído local em vez da regra geral.

10. Verificação de vazamento e leitura crítica do resultado

A acurácia de 0,90 ficou acima da faixa esperada (0,60 a 0,80), o que exigiu verificação antes de aceitar o resultado.

a) Vazamento de dados: aluno_id foi removido das features antes do treino, e nenhuma das colunas restantes é calculada a partir do rótulo ou só existe depois da recomendação ser definida. Não há vazamento identificado.

b) Por que o resultado ficou tão alto: a base é sintética — treino_recomendado foi gerado por uma função de regras (recomenda_treino, em gerar_dataset.py), com ~8% de ruído de rótulo aplicado de propósito. Isso define um teto esperado de desempenho: mesmo um modelo perfeito erraria aproximadamente os 8% de registros ruidosos. A seção 10 do notebook recalcula, para cada aluno, a recomendação "pura" da regra (sem ruído) e compara com o rótulo real e com a previsão do modelo.

Registros com ruído de rotulo na base toda (regra pura != rotulo real): 24 de 300 (8,0%)

No conjunto de teste: 6 erros do modelo em relacao ao rotulo real.

Desses 6 erros, 2 caem em registros com ruído de rotulo (33%); os outros 4 ocorrem em registros limpos.

Acuracia do modelo contra a regra pura (sem ruído): 0,9333

Leitura critica dos numeros acima: apenas 2 dos 6 erros (33%) coincidem com ruído de rotulo, ou seja, a hipotese de "o erro e quase todo ruído" nao se confirma por completo — a maioria dos erros (4 de 6) ocorre em registros limpos. A arvore recupera boa parte da regra de geracao (acuracia de 93,33% contra a regra pura, acima dos 90% contra o rotulo com ruído) e nao ha vazamento de dados, mas uma fracao do erro e genuina do modelo, concentrada no Treino D (recall 0,78 na secao 8), a classe com a execucao de regra identificada na secao 9. Consequencia para a AP2: o resultado de 0,90 mede majoritariamente a capacidade do modelo de recuperar a regra sintetica, mas com um residuo de erro proprio da arvore nas fronteiras entre classes, nao apenas do ruído injetado — e ainda assim nao diz nada sobre alunos reais de academia.

11. Comparação com o baseline e com a expectativa inicial

- baseline: 0,30
- modelo: 0,90
- diferença: +0,60

O modelo superou claramente o baseline e ficou acima da expectativa inicial (0,60–0,80). Diferente do esboço, essa diferença não foi apenas celebrada: foi investigada nas seções 8 a 10, concluindo que o resultado é consistente com a recuperação da regra de geração da base, dentro do teto definido pelo ruído de 8%.

12. Dificuldades encontradas

- As colunas categóricas vinham em texto, sendo necessário convertê-las com `get_dummies` antes do treinamento.
- A coluna `aluno_id` não poderia permanecer entre as features.
- Na entrega final, a principal dificuldade foi investigar por que a acurácia ficou acima do esperado, em vez de aceitar o número alto sem explicação — foi preciso reabrir o script gerador para confirmar a regra e o percentual de ruído.
- A principal lição desta etapa é que um resultado bom demais merece desconfiança e verificação, não comemoração imediata.

13. O que mudou em relação ao esboço

- O modelo e a divisão treino/teste foram mantidos; a justificativa do modelo foi reforçada com a análise de interpretabilidade (seção 9).
- Foram adicionadas matriz de confusão e F1 por classe (seção 8).
- Foi adicionada a extração das regras da árvore e comparação com a regra de geração (seções 9 e 10).
- Foi adicionada a verificação formal de vazamento de dados e a explicação quantitativa do resultado acima do esperado (seção 10).
- A conclusão foi reescrita para refletir essa leitura crítica.
- O link do repositório foi incluído nesta entrega, na seção "Artefato verificável" (ausente nas entregas anteriores).

14. Leitura crítica geral

O resultado faz sentido para o problema tal como ele foi definido: a base é sintética e gerada por regras conhecidas, então uma árvore de decisão consegue recuperar essas regras com alta acurácia. O resultado não comprova, porém, que o modelo funcionaria bem com alunos reais de academia — essa é a limitação mais importante do projeto nesta etapa, e deve orientar a discussão da AP2.

15. Backlog da AP2 (o que falta até o Encontro 11)

- [Feito] Treinar baseline e primeiro modelo (Atividade 9)
- [Feito] Matriz de confusão e F1 por classe
- [Feito] Verificar vazamento de dados e explicar o resultado via comparação com a regra de geração
- [Falta] Validação cruzada ($k=5$), conforme previsto no pipeline da Atividade 7
- [Falta] Discutir o que o resultado de 0,90 significa para alunos reais (vs. recuperação da regra sintética) — deve ser o centro da apresentação da AP2
- [Falta] Preparar os slides/apresentação da AP2

16. Organização, dificuldades e observações

A entrega foi organizada em 14 seções no notebook: carregamento da base tratada, expectativa inicial, divisão treino/teste, baseline, modelo, previsões, comparação com baseline, matriz de confusão e F1, regras da árvore, verificação de vazamento e leitura crítica, dificuldades, o que mudou em relação ao esboço, backlog da AP2 e conclusão.

O notebook foi executado do início ao fim no Google Colab e está disponível no repositório, com link incluído na seção "Artefato verificável" desta entrega.

17. Declaração de uso de IA

Uso de IA: Sim

Ferramentas utilizadas: ChatGPT e Claude (Anthropic) / IA generativa

Como foi usada:

- apoio na estruturação do código do notebook (matriz de confusão, F1 por classe, export_text e comparação das previsões com a regra de geração) e na organização do repositório;
- revisão da lógica de treino e avaliação do modelo;
- apoio na escrita do texto explicativo, na organização das seções da atividade e na revisão ortográfica;
- revisão de conceitos de Machine Learning aplicados ao problema, incluindo a interpretação do porquê a acurácia ficou acima da expectativa inicial.

Produção própria:

a definição do problema, a escolha da variável alvo, o critério de divisão treino/teste, a justificativa do modelo escolhido, a interpretação dos resultados, a leitura crítica de que a acurácia de 0,90 reflete a recuperação da regra sintética (e não o desempenho esperado com alunos reais) e a análise das dificuldades foram realizadas pelo aluno, com adaptação ao caso real do projeto.